MERN STACK

TRAINING TR-103 REPORT DAY 23

21 July 2026

Overview:

The twenty-third day of the MERN Stack training focused on handling GET and DELETE HTTP methods in Express.js and MongoDB. I learned how to retrieve data from the database using the GET method and remove existing records using the DELETE method. The session also covered the use of route parameters (`req.params`), `deleteOne()`, and fetching API data from React applications. Practical exercises helped us understand CRUD operations between the frontend and backend.

Learning Objectives:

- Understand the GET method.
- Learn how to retrieve data from MongoDB.
- Understand the DELETE method.
- Learn about route parameters (`req.params`).
- Use `deleteOne()` to remove documents.
- Connect React frontend with backend APIs.

MongoDB

GET Method

The GET method is an HTTP request method used to retrieve data from the server. It only fetches data and does not modify or delete any existing information.

Features:

- Retrieves data from the server.
- Does not change server data.
- Can be cached by browsers.
- Safe and idempotent.

Syntax (Express):

```
app.get(path, callback_function);
```

Fetching Data in React

The Fetch API is used to request data from the backend. After receiving the response, it is converted into JSON format and displayed in the React application.

Features:

- Retrieves API data.
- Uses asynchronous programming.
- Converts response into JSON.
- Displays dynamic data.

DELETE Method

The DELETE method is used to remove an existing resource or document from the server or database.

Features:

- Deletes existing records.
- Requires the unique ID of the document.
- Does not create duplicate operations.

- Commonly used in CRUD applications.

Syntax (Express):

```
app.delete(path, callback_function);
```

Route Parameters (`req.params`)

Route parameters are dynamic values passed through the URL. They are accessed using the `req.params` object.

Features:

- Retrieves values from the URL.
- Commonly used for IDs.
- Helps identify a specific document.

Example:

```
req.params.id
```

deleteOne() Method

The `deleteOne()` method is a Mongoose function used to remove a single document that matches the specified condition.

Features:

- Deletes one matching document.
- Uses a condition or document ID.
- Returns the deletion status.

Syntax:

```
Model.deleteOne(condition);
```

API Response

After a delete operation, the server sends a JSON response to indicate whether the operation was successful.

Common Responses:

- **statuscode: 1** – Record deleted successfully.
- **statuscode: 0** – Deletion failed.

Task –1



```
app.get("/api/getuser", async (req, res) => {
    const data = await usermodel.find();

    console.log(data); // <-- Add this

    if (data.length > 0) {
        res.send({
            statuscode: 1,
            alldata: data
        });
    } else {
        res.send({
            statuscode: 0,
            msg: "No data found"
        });
    }
});
```



```
app.post("/api/postuser",async(req,res)=>{
const findser=await usermodel.findOne({Email:req.body.mail});
if(findser){
    res.send({statuscode:2,msg:"Mail aleready exist"})
}else{

    const data=await usermodel({
        Name:req.body.name,
        Email:req.body.mail,
        Age:req.body.age
    });
    const savedata=await data.save();
    if(savedata){
        res.send({statuscode:1,msg:"Success"})
    }else{
        res.send({statuscode:0,msg:"error"})
    }
}
})
```

Output:





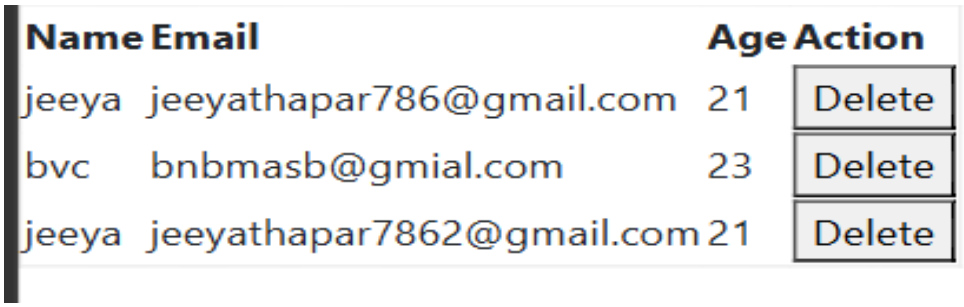
TASK –2 :

```
app.delete("/api/deluser/:id", async (req, res) => {
  console.log("ID:", req.params.id);

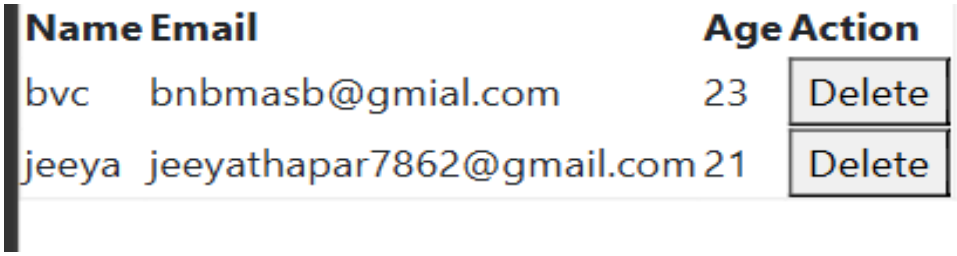
  const finduser = await usermodel.deleteOne({ _id: req.params.id });

  console.log(finduser);

  if (finduser.deletedCount > 0) {
    res.send({ statuscode: 1 });
  } else {
    res.send({ statuscode: 0 });
  }
});
```







Conclusion:

By the end of Day 23, I gained a clear understanding of implementing GET and DELETE operations in a MERN Stack application. I learned how to retrieve data from MongoDB, display it in React, delete records using `deleteOne()`, and use route parameters to identify specific documents, strengthening my knowledge of CRUD operations.